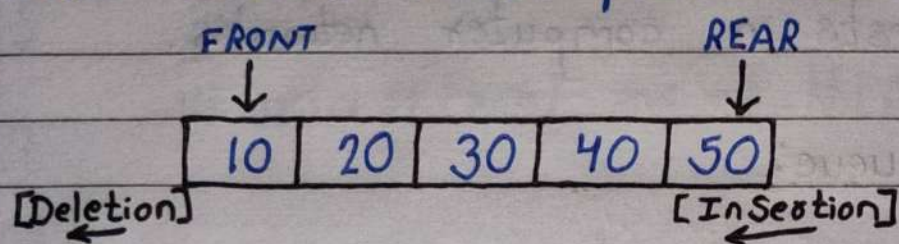


Queue

A Queue is a linear data structure that follows the FIFO (First In, First Out) principle. The element inserted first is removed first.

In queue, insertion takes place at the REAR, while deletion takes place at the FRONT.



Operations of Queue:

1. Enqueue: Inserts an element at the rear of the queue.
2. Dequeue: Removes an element from the front of the queue.
3. Peek: Returns the front element without removing it.
4. isEmpty: Checks whether the queue is empty.
5. isFull: Checks whether an array-based queue is full.

Applications of Queue:

- CPU scheduling.
- Printer job scheduling.
- Breadth First Search (BFS) Algorithm.
- Handling requests in computer networks.

Types of Queue:

1. Linear/Simple Queue
2. Circular Queue
3. Double-Ended Queue (Deque)
4. Priority Queue

Implementation of Queue Using Array

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int queue[MAX], front = -1, rear = -1;
```

```
int isEmpty()
```

```
{  
    return (front == -1 || front > rear);  
}
```

```
int isFull()
```

```
{  
    return (rear == MAX - 1);  
}
```

```
void enqueue(int value)
```

```
{  
    if (isFull())  
    {
```

```
        printf("Queue is full \n");  
        return;  
    }
```

```
    if (front == -1)  
        front = 0;
```

```
    queue[++rear] = value;
```

```
}
```

~~void~~

int dequeue()

{

if (isEmpty())

{

printf("Queue is empty \n");

return -1;

}

int deleted_value = queue[front];

if (front > rear)

front = rear = -1;

else

front++;

return deleted_value;

}

int peek()

{

if (isEmpty())

{

printf("Queue is empty \n");

return -1;

}

return queue[front];

}

```

void display()
{
    if (isEmpty())
    {
        printf("Queue is empty \n");
        return;
    }

    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);

    printf("\n");
}

```

```

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);

    display();

    return 0;
}

```

Output:

10 20 30 40

Implementation of Queue Using Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
struct Node *front = NULL, *rear = NULL;
```

```
int isEmpty()
```

```
{
```

```
    return (front == NULL);
```

```
}
```

```
void enqueue(int value)
```

```
{
```

```
    struct Node *new_node;
```

```
    new_node = malloc(sizeof(struct Node));
```

```
    new_node->data = value;
```

```
    new_node->next = NULL;
```

```
    if (front == NULL)
```

```
        front = rear = new_node new_node;
```

```
    else
```

```
    {
```

```
        rear->next = new_node;
```

```
        rear = new_node;
```

```
    }
```

```
}
```

```
void dequeue()
{
    struct Node *temp;
    if (isEmpty())
    {
        printf("Queue Underflow\n");
        return;
    }

    temp = front;
    front = front → next;

    if (front == NULL)
        rear = NULL;
    printf("Deleted : %d\n", temp → data);
    free(temp);
}
```

```
void display()
{
    if (isEmpty())
    {
        printf("Queue is empty\n");
        return;
    }
```

```
    struct Node *temp = front;
```

```
while (temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
```

```
printf("\n");
}
```

```
int main()
{
```

```
    enqueue(10);
    enqueue(20);
    enqueue(30);
```

```
    display();
    struct Node *next_node;
    while (front != NULL)
    {
```

```
        front next_node = front->next;
        free(front);
        front = next_node;
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

10 20 30

Circular Queue

A circular queue is a type of queue in which the last position is connected to the first position, forming a circle.

It follows the FIFO principle and solves the problem of wasted space in a linear queue.

In this, rear or front is increment using a modulo operator '%'.
$$(rear + 1) \% MAX$$
$$(front + 1) \% MAX$$

$$(rear + 1) \% MAX$$

$$(front + 1) \% MAX$$

Here, modulo allows the index to return to 0 index after reaching the last position.

In this, queue is full or not (for array based implementation) is check using,

$$[(rear + 1) \% MAX == front]$$

Program:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int queue[MAX], front = -1, rear = -1;
```

```
void void enqueue(int value)
```

```
{
```

```
    if ((rear + 1) % MAX == front)
```

```
    {
```

```
        printf("Queue Overflow \n");
```

```
        return;
```

```
    }
```

```
    if (front == -1)
```

```
        front = 0;
```

```
    rear = (rear + 1) % MAX;
```

```
    queue[rear] = value;
```

```
}
```

```
void dequeue()
```

```
{
```

```
    if (front == -1)
```

```
    {
```

```
        printf("Queue Underflow \n");
```

```
        return;
```

```
    }
```

```
    printf("Deleted: %d \n", queue[front]);
```

```
if (front == rear)
    front = rear = -1;
else
    front = (front + 1) % MAX;
```

```
}
```

```
void display()
```

```
{
```

```
    int i;
```

```
    if (front == -1)
```

```
    {
```

```
        printf("Queue is empty \n");
        return;
```

```
    }
```

```
    i = front;
```

```
    while (1)
```

```
    {
```

```
        printf("%d ", queue[i]);
```

```
        if (i == rear)
```

```
            break;
```

```
        i = (i + 1) % MAX;
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main()  
{
```

```
    enqueue(10);  
    enqueue(20);  
    enqueue(30);  
    enqueue(40);  
    enqueue(50);
```

```
    printf("Queue: ");  
    display();
```

```
    return 0;
```

```
}
```

Output:

Queue: 10 20 30 40 50

Double-Ended Queue (Deque)

A Double-Ended Queue or Deque is a type of queue in which insertion and deletion can be performed at both the front and rear ends. Unlike a simple queue, a deque allows operations from either end.

Types of Deque:

1. Input-Restricted Deque: Insertion is allowed at only one end, but deletion is allowed at both ends.
2. Output-Restricted Deque: Deletion is allowed at only one end, but insertion is allowed at both ends.

Basic Operations of Deque:

1. insertFront : Inserts an element at the front.
2. insertRear: Inserts an element at the rear.
3. deleteFront : Deletes an element from the front.
4. deleteRear: Deletes an element from the rear.

Program:

```
#include <stdio.h>
#define MAX 10
```

```
int deque[MAX], front = -1, rear = -1;
```

```
void insertFront(int value)
{
```

```
    if (front == 0)
    {
```

```
        printf("No space at front \n");
        return;
    }
```

```
    if (front == -1)
```

```
        front = rear = 0;
```

```
    else
```

```
        front --;
```

```
    deque[front] = value;
```

```
}
```

```
void insertRear(int value)
```

```
{
```

```
    if (rear == MAX - 1)
    {
```

```
        printf("No space at rear \n");
        return;
    }
```

```
}
```

```
if (front == -1)
    front = 0;
```

```
    deque[++rear] = value;
}
```

```
void deleteFront()
```

```
{
    if (front == -1)
```

```
{
        printf("Deque Underflow \n");
        return;
    }
```

```
    printf("Deleted: %d \n", deque[front]);
```

```
    if (front == rear)
```

```
        front = rear = -1;
```

```
    else
```

```
        front++;
```

```
}
```

```
void deleteRear()
```

```
{
    if (rear == -1)
```

```
{
        printf("Deque Underflow \n");
        return;
    }
```

```
    printf("Deleted: %d \n", deque[rear]);
```

```

        if (front == rear)
            front = rear = -1;
        else
            rear--;
    }

```

```

void display()
{
    if (front == -1)
    {
        printf("Deque is empty \n");
        return;
    }

```

```

        for (int i = front; i <= rear; i++)
            printf("%d ", deque[i]);

        printf("\n");
    }

```

```

int main()
{
    insertRear(20);
    insertRear(30);
    insertFront(10);
    display();
    return 0;
}

```

Output:

10 20 30

Priority Queue

A Priority Queue is a type of queue in which each element is assigned a priority. Unlike a standard queue, a priority queue processes and removes elements based on their priority. If two elements have the same priority, they are processed according to FIFO.

Types of Priority Queue:

1. Ascending (Min) Priority Queue

The element with the smallest priority value is deleted first.

(Max)

2. Descending $\wedge$ Priority Queue

The element with the largest priority value is deleted first.

Program:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
struct Element
```

```
{  
    int data;  
    int priority;  
};
```

```
struct Item Element queue[MAX];
```

```
int rear = -1;
```

```
void enqueue (int value, int priority)  
{
```

```
    if (rear == MAX-1)  
    {
```

```
        printf("Queue Overflow\n");  
        return;
```

```
    }
```

```
    rear++;
```

```
    queue[rear].data = value;
```

```
    queue[rear].priority = priority;
```

```
}
```

```
void dequeue()
```

```
{
```

```
    if (rear == -1)
```

```
    {
```

```
        printf("Queue Underflow\n");  
        return;
```

```
    }
```

```
    int pos = 0;
```

```
    for (int i = 1; i <= rear; i++)
```

```
    {
```

```
        if (queue[i].priority < queue[pos].priority)  
            pos = i;
```

```
    }
```

```
    printf("Deleted : %d\n", queue[pos].data);
```

```
    for (int i = pos; i < rear; i++)
```

```
        queue[i] = queue[i+1];
```

```
    rear--;
```

```
}
```

```
void display()
```

```
{
```

```
    if (rear == -1)
```

```
    {
```

```
        printf("Queue is empty\n");  
        return;
```

```
    }
```

```
for (int i = 0; i <= rear; i++)  
    printf("%d (%d) ", queue[i].data,  
           queue[i].priority);
```

```
    printf("\n");  
}
```

```
int main()  
{
```

```
    enqueue(10, 3);
```

```
    enqueue(20, 1);
```

```
    enqueue(30, 2);
```

```
    printf("Priority Queue: ");  
    display();
```

```
    return 0;
```

```
}
```

Output:

Priority Queue: 10(3) 20(1) 30(2)